

Dream Paintings

■ Project Identity

****Project Name:**** Dream Paintings

****Tagline:**** A premium digital marketplace for discovering, submitting, and managing original fine art.

****Purpose:****

Dream Paintings is designed to connect artists, buyers, and platform administrators through a full-stack online art marketplace. It solves the problem of decentralized discovery, approval workflow, and transaction tracking for original paintings by providing a dedicated, polished platform where:

- artists can submit and manage artwork,
- buyers can browse gallery collections and purchase pieces,
- admins can review submissions, manage orders, and tune homepage content.

The project bridges creative discovery with a simple commerce experience, while surfacing artist profiles, trending works, and editorial curation.

****Vision:****

To evolve into a trusted, scalable art-commerce platform where curated artists can showcase premium work globally, and gallery administrators can manage creative inventory, marketplace reputation, and order fulfillment from a single backend.

****Target Audience:****

- Independent visual artists seeking an online gallery storefront.
- Art collectors and buyers searching for curated paintings.
- Marketplace operators and admins who need a moderated submission, order, and content control interface.

****Use Cases:****

- An emerging painter submits a new canvas for approval and receives admin feedback.
- A buyer searches by category, likes a painting, comments on it, and completes a purchase.
- An admin reviews pending submissions, approves or rejects work, and promotes featured pieces.
- The marketing team updates homepage text sections, testimonials, and featured collections without code changes.

■ Tech Stack

Frontend

- React 19 with functional components and hooks
- Vite for fast development and production builds
- Tailwind CSS 4 for utility-first styling
- Framer Motion for page and component animation
- `@react-three/fiber` and `@react-three/drei` for 3D hero visuals
- `react-router-dom` for client-side routing
- `axios` for API communication
- `react-hot-toast` for toast notifications

- `lucide-react` for icons
- `@tsparticles/react` for particle background effects

Backend

- Node.js with Express 5 web server
- PostgreSQL via `pg` client library
- JWT authentication with `jsonwebtoken`
- Password hashing with `bcryptjs`
- File upload handling with `multer` and Cloudinary storage
- `express-validator` installed but controllers perform basic manual checks
- CORS enabled for local development and optional ngrok domains

Database

- PostgreSQL (SQL)
- Runtime schema creation and migration via `backend/config/db.js`
- Table-driven data model for users, paintings, orders, notifications, and admin settings

Authentication system

- JWT tokens stored in `localStorage`
- `Authorization: Bearer` header on all protected API calls
- Role-based access control for `user`, `artist`, and `admin`

External APIs and services

- Cloudinary for image uploads, avatars, payment proofs, QR codes
- Optional frontend `VITE\_API\_URL` proxy for backend integration

Hosting / deployment environment

- Frontend built with Vite; production bundle served by any static host
- Backend deployable to Node hosting like Railway, Heroku, DigitalOcean, or AWS
- PostgreSQL hosted externally via a service such as Neon, Supabase, or any managed provider

■■ Architecture Overview

High-level architecture

```
[Browser React SPA] <--> [Express API Server] <--> [PostgreSQL Database]
                        |
                        +--> [Cloudinary Image Storage]
```

Client ↔ Server interaction

- The React SPA sends requests to `/api/\*` endpoints.
- Axios automatically attaches JWT tokens from `localStorage`.
- Backend middleware validates user tokens and authorizes routes.
- Controllers execute business logic and query PostgreSQL.
- Images are uploaded directly through Cloudinary-storage middleware.

Data flow explanation

1. User authenticates via `/api/auth/login` or `/api/auth/register`.
2. Client stores JWT token and fetches profile via `/api/auth/me`.
3. Buyers browse `/api/paintings` and view details with `/api/paintings/:id`.
4. Artists upload images and metadata to `/api/paintings`.
5. Admin reviews pending paintings and updates approval status.
6. Orders are created through `/api/orders` and managed by admins.
7. Notifications are inserted into the notifications table and fetched by clients.

Offline/online capability

- No explicit offline-first mode is implemented.
- The app is optimized for online SPA usage with server-driven data.

■ Folder Structure

Root

- `backend/` — Express API server, database config, routes, controllers
- `frontend/` — React SPA source and build configuration
- `README.md` — project documentation
- `START.bat`, `START\_FULL.bat`, `NGROK-START.bat` — local run helpers
- `render.yaml`, `railway.toml`, `vercel.json` — deployment metadata

backend/

- `server.js` — main Express application entrypoint
- `package.json` — backend dependencies and scripts
- `createAdmin.js` — utility script to create or reset an admin account
- `config/db.js` — PostgreSQL connection, schema creation, migrations
- `config/cloudinary.js` — Cloudinary setup and Multer storage configuration
- `controllers/` — business logic for auth, paintings, admin, and orders
- `middleware/auth.js` — JWT validation and role guard
- `routes/` — route declarations for auth, paintings, admin, notifications, orders

frontend/

- `package.json` — frontend dependencies and scripts
- `vite.config.js` — Vite config and local API proxying
- `src/main.jsx` — SPA bootstrap

- ``src/App.jsx`` — routing and global providers
- ``src/context/AuthContext.jsx`` — user auth state and session handling
- ``src/utils/api.js`` — Axios instance with interceptors
- ``src/components/`` — reusable UI building blocks
- ``src/pages/`` — page-level views
- ``src/pages/admin/`` — admin dashboard and management sections

Important files explained

- ``backend/config/db.js`` — auto-creates tables and alters schema on startup
- ``backend/config/cloudinary.js`` — exports upload middleware for paintings, avatars, QR codes, and proofs
- ``frontend/src/utils/api.js`` — attaches JWT and handles 401 responses
- ``frontend/src/context/AuthContext.jsx`` — loads current user and exposes login/register/logout
- ``frontend/src/App.jsx`` — defines public, protected, and admin-only routes

■ ■ Features Breakdown

Authentication & User Management

- Purpose: control access and separate buyers, artists, and admins.
- Components involved: ``AuthContext``, ``Login``, ``Register``, ``ProtectedRoute``, backend auth routes.
- Backend logic: ``authController`` validates credentials, hashes passwords, issues JWT.
- Database usage: stored in ``users`` table with ``role``, ``avatar_url``, ``bio``, ``is_banned``, ``is_master_artist``.
- Edge cases: duplicate emails are rejected; invalid login returns 401.

Painting Discovery

- Purpose: display artwork for browsing, searching, and filtering.
- Components involved: ``Home``, ``Gallery``, ``PaintingCard``, ``PaintingDetail``, ``CategoryGrid``.
- Backend logic: ``paintingController.getPaintings()`` supports search, category, pagination, featured filtering, sorting.
- Database usage: ``paintings`` table, ``likes`` and ``comments`` joined for counts.
- Edge cases: only approved and non-deleted paintings are shown by default; empty states handled.

Painting Submission Workflow

- Purpose: artist upload pipeline with approval checks.
- Components involved: ``SubmitPainting``, ``admin/PendingSection``, ``admin/PaintingsSection``.
- Backend logic: ``createPainting`` saves uploaded images to Cloudinary and sets status to ``pending`` for artists, ``approved`` for admins.
- Database usage: ``paintings`` table stores image URLs, metadata, status, admin messages.
- Edge cases: missing image or missing metadata returns a 400.

Like & Comment System

- Purpose: social proof and interaction around artwork.
- Components involved: ``PaintingDetail``, ``CommentsSection``, ``Notifications``.
- Backend logic: ``toggleLike``, ``addComment``, ``getComments``, ``deleteComment``.

- Database usage: ``likes`` table for uniqueness, ``comments`` table with user references.
- Edge cases: duplicate likes are toggled off; only comment owners or admins can delete comments.

Order & Purchase Flow

- Purpose: manage payment orders and purchase records.
- Components involved: ``CustomerDetails``, ``PaymentPage``, ``OrderSuccess``, ``OrderHistory``.
- Backend logic: ``createOrder``, ``getMyOrders``, ``getOrderById``, ``updateOrderStatus``, ``purchasePainting``.
- Database usage: ``orders`` and ``sales`` store order lifecycle, transaction proof, status updates.
- Edge cases: users cannot create duplicate active orders for same painting; purchase route verifies painting approval.

Admin Dashboard

- Purpose: central control panel for operations, content, and metrics.
- Components involved: ``AdminDashboard``, ``OverviewSection``, ``UsersSection``, ``PendingSection``, ``SalesSection``, ``OrdersSection``, ``CommentsSection``, ``NotifySection``, ``HomepageEditor``, ``ActivitySection``.
- Backend logic: admin-only routes under ``/api/admin/*`` with role validation.
- Database usage: ``activity_logs``, ``settings``, ``testimonials``, ``notifications``, ``orders``, ``users``, ``paintings``.
- Edge cases: admin actions log operations and guard deletion/ban flows.

Homepage CMS

- Purpose: let admin tune landing page sections and testimonial content.
- Components involved: ``HomepageEditor``, homepage sections in ``Home.jsx``.
- Backend logic: ``getHomepageConfig``, ``saveHomepageConfig``, ``getTestimonials``, ``createTestimonial``, ``updateTestimonial``, ``deleteTestimonial``, ``getPublicHomepageData``.
- Database usage: ``settings`` stores key/value homepage content; ``testimonials`` stores reviews.
- Edge cases: fallback defaults exist when settings are missing.

■ Authentication & User System

Signup/login flow

1. ``POST /api/auth/register`` with ``name``, ``email``, ``password``, optional ``role``.
2. Backend checks existing user, hashes password, inserts user, issues JWT.
3. Frontend stores JWT in ``localStorage`` and populates auth context.
4. ``POST /api/auth/login`` validates credentials and returns token + user object.
5. ``GET /api/auth/me`` refreshes session after page load.

Roles

- ``user`` — buyer and comment author.
- ``artist`` — painting submitter with profile and submission workflows.
- ``admin`` — full dashboard and moderation privileges.

Session / JWT handling

- JWT token expires in 7 days.
- Stored in browser `localStorage`.
- Axios interceptor attaches `Authorization: Bearer`.
- `auth` middleware verifies JWT and sets `req.user`.
- `requireRole(...)` protects role-based routes.

Security approach

- Password hashing with bcrypt.
- Role-based middleware on protected routes.
- CORS configured for localhost and ngrok domains.
- HTTP 401 responses clear tokens client-side.

■■ Database Design

DB type

- PostgreSQL relational database.

Main tables

`users`

- `id` SERIAL PRIMARY KEY
- `name` VARCHAR(100)
- `email` VARCHAR(150) UNIQUE
- `password` VARCHAR(255)
- `role` VARCHAR(20) CHECK IN ('user', 'artist', 'admin')
- `avatar\_url` TEXT
- `bio` TEXT
- `is\_banned` BOOLEAN
- `is\_master\_artist` BOOLEAN
- `created\_at` TIMESTAMP DEFAULT NOW()

`paintings`

- `id` SERIAL PRIMARY KEY
- `title` VARCHAR(200)
- `description` TEXT
- `image\_url` TEXT
- `price` DECIMAL(10,2)
- `category` VARCHAR(100)
- `artist\_id` INTEGER REFERENCES users(id)
- `status` VARCHAR(20) CHECK IN ('pending', 'approved', 'rejected', 'sold')
- `admin\_message` TEXT
- `views` INTEGER DEFAULT 0
- `is\_featured` BOOLEAN DEFAULT FALSE

- `is\_trending` BOOLEAN DEFAULT FALSE
- `deleted\_at` TIMESTAMP
- `discount\_percent` DECIMAL(5,2)
- `offer\_start` TIMESTAMP
- `offer\_end` TIMESTAMP
- `created\_at` TIMESTAMP DEFAULT NOW()

`comments`

- `id` SERIAL PRIMARY KEY
- `user\_id` INTEGER REFERENCES users(id)
- `painting\_id` INTEGER REFERENCES paintings(id)
- `text` TEXT
- `created\_at` TIMESTAMP DEFAULT NOW()

`likes`

- `user\_id` INTEGER REFERENCES users(id)
- `painting\_id` INTEGER REFERENCES paintings(id)
- PRIMARY KEY `(user\_id, painting\_id)`

`orders`

- `id` SERIAL PRIMARY KEY
- `order\_id` VARCHAR(20) UNIQUE
- `user\_id` INTEGER REFERENCES users(id)
- `painting\_id` INTEGER REFERENCES paintings(id)
- `amount` DECIMAL(10,2)
- `status` VARCHAR(20) CHECK IN ('pending','approved','rejected')
- `payment\_proof` TEXT
- `transaction\_id` VARCHAR(200)
- `full\_name`, `mobile`, `email`, `address`, `city`, `state`, `pincode`, etc.
- `rejection\_reason` TEXT
- `created\_at` TIMESTAMP DEFAULT NOW()

`sales`

- `id` SERIAL PRIMARY KEY
- `user\_id` INTEGER REFERENCES users(id)
- `painting\_id` INTEGER REFERENCES paintings(id)
- `amount` DECIMAL(10,2)
- `status` VARCHAR(20) CHECK IN ('pending','completed','cancelled')
- `created\_at` TIMESTAMP DEFAULT NOW()

`notifications`

- `id` SERIAL PRIMARY KEY
- `user\_id` INTEGER REFERENCES users(id)
- `message` TEXT
- `is\_read` BOOLEAN DEFAULT FALSE
- `created\_at` TIMESTAMP DEFAULT NOW()

`activity\_logs`

- `id` SERIAL PRIMARY KEY
- `admin\_id` INTEGER REFERENCES users(id)
- `action` VARCHAR(200)
- `target\_type` VARCHAR(50)
- `target\_id` INTEGER

- `details` TEXT
- `created\_at` TIMESTAMP DEFAULT NOW()

`settings`

- `key` VARCHAR(100) PRIMARY KEY
- `value` TEXT
- `updated\_at` TIMESTAMP DEFAULT NOW()

`testimonials`

- `id` SERIAL PRIMARY KEY
- `name` VARCHAR(100)
- `role` VARCHAR(100)
- `quote` TEXT
- `avatar\_url` TEXT
- `is\_active` BOOLEAN
- `sort\_order` INTEGER
- `created\_at` TIMESTAMP DEFAULT NOW()

Relationships

- `users` → `paintings` via `artist\_id`
- `users` → `comments` and `likes`
- `paintings` → `orders` and `sales`
- `admins` manage `activity\_logs`, `settings`, and `testimonials`

Sample schema snippet

```
CREATE TABLE users (...);
CREATE TABLE paintings (...);
CREATE TABLE comments (...);
CREATE TABLE likes (...);
CREATE TABLE orders (...);
CREATE TABLE notifications (...);
```

■ API Documentation

Auth

- `POST /api/auth/register`
- body: `{ name, email, password, role }`
- response: `{ token, user }`
- `POST /api/auth/login`
- body: `{ email, password }`
- response: `{ token, user }`
- `GET /api/auth/me`
- headers: `Authorization: Bearer `
- response: user details
- `PUT /api/auth/profile`
- multipart/form-data: `avatar`, `name`, `bio`

- response: updated user

Paintings

- `GET /api/paintings`
- query: `search`, `category`, `page`, `limit`, `featured`, `sort`
- response: `{ paintings, total, page, limit }`
- `GET /api/paintings/:id`
- response: painting details with artist metadata
- `GET /api/paintings/artist/:artistId`
- response: artist portfolio
- `POST /api/paintings`
- protected: `artist|admin`
- multipart/form-data: `title`, `description`, `price`, `category`, `image`
- response: created painting
- `PUT /api/paintings/:id`
- protected: owner or admin
- body: update fields like `title`, `description`, `price`, `status`
- `DELETE /api/paintings/:id`
- protected: owner or admin
- `POST /api/paintings/:id/like`
- protected: user
- response: `{ liked, likes\_count }`
- `GET /api/paintings/:id/comments`
- response: comment list
- `POST /api/paintings/:id/comments`
- protected: user
- body: `{ text }`
- `DELETE /api/paintings/:id/comments/:commentId`
- protected: comment owner or admin
- `POST /api/paintings/:id/purchase`
- protected: user
- creates sale record and notifications

Orders

- `POST /api/orders`
- protected: user
- multipart/form-data: `painting\_id`, `full\_name`, `mobile`, `email`, `address`, `city`, `state`, `pincode`, optional `payment\_proof`, `transaction\_id`
- response: created order
- `GET /api/orders/my`
- protected: user
- response: buyer order history
- `GET /api/orders/:id`
- protected: user
- response: order detail with painting and artist
- `GET /api/notifications/qr`
- public: returns payment QR URL

Admin

- All admin routes require `Authorization` and `admin` role.
- `GET /api/admin/stats`
- `GET /api/admin/users`
- `DELETE /api/admin/users/:id`
- `PUT /api/admin/users/:id/ban`
- `PUT /api/admin/users/:id/role`
- `PUT /api/admin/users/:id/master-artist`
- `GET /api/admin/master-artists`
- `GET /api/admin/paintings`
- `GET /api/admin/pending-paintings`
- `PUT /api/admin/paintings/:id`
- `DELETE /api/admin/paintings/:id`
- `PUT /api/admin/paintings/:id/restore`
- `POST /api/admin/paintings/bulk`
- `GET /api/admin/comments`
- `DELETE /api/admin/comments/:id`
- `GET /api/admin/sales`
- `GET /api/admin/activity-logs`
- `POST /api/admin/notify`
- `GET /api/admin/notifications`
- `PUT /api/admin/notifications/read`
- `GET /api/admin/orders`
- `PUT /api/admin/orders/:id/status`
- `POST /api/admin/qr`
- `GET /api/admin/homepage/config`
- `POST /api/admin/homepage/config`
- `GET /api/admin/homepage/testimonials`
- `POST /api/admin/homepage/testimonials`
- `PUT /api/admin/homepage/testimonials/:id`
- `DELETE /api/admin/homepage/testimonials/:id`

■ UI/UX System

Design style

- Modern premium art marketplace aesthetic
- Glassmorphism cards, gradient accents, and red/white brand palette
- 3D hero scene via React Three Fiber

Responsiveness

- Mobile-first layout with responsive grid breakpoints
- Adaptive admin sidebar for desktop and mobile
- Interactive CTA and pill navigation for filters

Animations/interactions

- Framer Motion page reveals and hover transitions
- Button hover animations and toast feedback
- Drag-and-drop image uploader in submission flow
- Hover controls for comment deletion and action buttons

Theme system

- Admin panel supports a toggle between light/dark shell mode
- Frontend uses bright art marketplace branding with consistent UX

■ Data Handling

Data storage

- Auth token persisted in `localStorage`
- User object stored in React context for session state
- Upload assets stored in Cloudinary, referenced by URL in DB

Data fetching

- Axios instance with request interceptor to attach JWT
- Protected calls automatically redirect to login on 401
- Gallery and homepage endpoints fetch paged and filtered data

State management

- Local React state and context handle user session and page state
- No external state library required

Caching strategy

- No dedicated caching layer is implemented
- Server-side query filtering and pagination reduce payloads
- Browser caching handled by the SPA runtime and modern browsers

■ Security Practices

Data protection

- Passwords hashed with bcrypt before storage
- Sensitive keys kept in backend `.env`
- Cloudinary image upload paths isolate media storage

Authentication security

- JWTs signed with secret and 7-day expiration
- Protected API routes guard unauthorized access
- Role-based middleware enforces admin-only resources

Input validation

- Controllers perform presence checks and role validation
- File uploads restricted by Cloudinary accepted formats

Possible vulnerabilities

- `localStorage` token storage is susceptible to XSS if scripts are injected
- No rate-limiting middleware is configured
- No CSRF-specific defenses present
- Some request validation is manual instead of using a full schema validator

■ Performance Optimization

Frontend optimizations

- Vite-powered fast bundling and HMR for development
- Client-side pagination avoids loading all paintings at once

Backend / DB optimization

- SQL queries aggregate likes/comments counts via joins
- `LIMIT`/`OFFSET` used for paginated admin lists and gallery data

Recommended improvements

- Add backend caching for homepage and gallery responses
- Add database indexes on searchable fields such as `paintings.title`, `paintings.category`, `users.email`
- Use lazy-loaded route components for admin sections
- Move static frontend assets to a CDN for production

■ Deployment Guide

Local setup

1. Clone repository.

2. Install backend dependencies:

```
cd backend  
npm install
```

3. Install frontend dependencies:

```
cd ../frontend  
npm install
```

Environment variables

Create `backend/.env` with:

```
PORT=5000  
DATABASE_URL=postgresql://user:pass@host/dbname?sslmode=require  
JWT_SECRET=your_super_secret_key  
CLOUDINARY_CLOUD_NAME=your_cloud_name  
CLOUDINARY_API_KEY=your_api_key  
CLOUDINARY_API_SECRET=your_api_secret  
FRONTEND_URL=http://localhost:5173
```

Start in development

- Backend: `npm run dev` from `backend/`
- Frontend: `npm run dev` from `frontend/`

Production build

- Frontend: `npm run build` in `frontend/`
- Backend: `npm start` in `backend/`

Deployment notes

- Ensure `DATABASE\_URL` includes SSL settings if required by your hosting provider.
- Set `VITE\_API\_URL` in the frontend environment if your backend is hosted separately.
- Use managed Postgres and Cloudinary credentials for production.

■ Testing

- No dedicated automated test suite is included in this repository.
- Potential future tooling: Jest + React Testing Library for frontend, Supertest for backend route testing.

■■■ Admin Panel

Admin features

- Dashboard metrics for users, artists, paintings, sales, pending approvals, comments
- User management: ban/unban, delete, role promotion, master artist assignment
- Painting moderation: approve/reject, feature, trend, soft-delete, restore

- Order control: view pending orders, approve/reject, payment QR upload
- Content management: homepage section text, testimonials, QR code settings
- Notifications and activity logs

Controls

- Hidden admin route: ``/super-admin-portal-xyz`` and ``/super-admin-secret``
- Admin-only middleware protects all ``/api/admin/*`` endpoints
- Admin logs stored in ``activity_logs`` for audit tracking

Data management capabilities

- Bulk painting actions: approve, reject, delete, feature, trending
- Notification broadcast by user, role, or all users
- Homepage CMS persistence via ``settings`` table

■ **Special Features**

- Multi-role marketplace with artist submission approval workflow
- Embedded homepage CMS for marketing copy and featured content
- Direct Cloudinary uploads for assets and payment proofs
- Hidden admin entrypoint for production-style access control
- Real-time-style gallery with trending and featured collections

■ **Future Improvements**

Suggested upgrades

- Implement a formal migration system with tools like Knex, Prisma, or Flyway
- Add automated tests and CI pipelines
- Move auth token to secure HTTP-only cookies
- Add server-side validation with ``zod`` or ``joi``
- Introduce Redis caching for homepage and gallery queries
- Add WebSocket notifications for real-time admin and buyer alerts
- Convert admin dashboard sections to lazy-loaded dynamic imports

Scalability ideas

- Split backend into microservices for auth, catalog, and order operations
- Deploy frontend and backend separately with a CDN and API gateway
- Add search indexing for faster artwork discovery
- Add payment gateway integration for real transactions

■ Screenshots / Demo

> Add screenshots and demo links here once available.

Notes from architecture review

- The current implementation is a solid full-stack foundation, but it should move away from runtime schema migrations to a proper migration pipeline.
- The admin route is obscured rather than fully protected by discovery-resistant access controls; a stronger RBAC and secret management layer is recommended.
- The project is production-ready as an MVP, but security hardening and automated testing should be prioritized before live deployment.